Proje

April 19, 2026

1 Chicago Ev Fiyatları Regresyon Analizi Projesi

Projeyi Hazırlayan: Enes Talha Aksoy

1.1 İlk Olarak Kullanılacak Kütüphaneleri Ekleyelim

[11]:
```
import numpy as np
import pandas as pd
import seaborn as sb
import matplotlib.pyplot as mt
import statsmodels.api as sm
import statsmodels.formula.api as smf
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import scipy.stats as stats
import pylab
from scipy.stats import shapiro
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_percentage_error
from sklearn.preprocessing import scale
from sklearn.decomposition import PCA
from sklearn import model_selection
from sklearn.model_selection import KFold
from sklearn.cross_decomposition import PLSRegression
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.linear_model import LassoCV
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import BaggingRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor
```

1.2 Şimdi Data'yı Yükleyelim

[12]:
```
data = pd.read_csv("C:/Users/asus/Downloads/realest.csv")
```

[13]:
```
data.head()
```

[13]:

	Price	Bedroom	Space	Room	Lot	Tax	Bathroom	Garage	Condition
0	53.0	2.0	967.0	5.0	39.0	652.0	1.5	0.0	0.0
1	55.0	2.0	815.0	5.0	33.0	1000.0	1.0	2.0	1.0
2	56.0	3.0	900.0	5.0	35.0	897.0	1.5	1.0	0.0
3	58.0	3.0	1007.0	6.0	24.0	964.0	1.5	2.0	0.0
4	64.0	3.0	1100.0	7.0	50.0	1099.0	1.5	1.5	0.0

[4]:
```
data.columns
```

[4]:
```
Index(['Price', 'Bedroom', 'Space', 'Room', 'Lot', 'Tax', 'Bathroom', 'Garage',
       'Condition'],
      dtype='object')
```

[5]:
```
data.dtypes
```

[5]:
```
Price        float64
Bedroom      float64
Space        float64
Room         float64
Lot          float64
Tax          float64
Bathroom     float64
Garage       float64
Condition    float64
dtype: object
```

[6]:
```
data.shape
```

[6]:
```
(157, 9)
```

[7]:
```
data.describe()
```

[7]:

	Price	Bedroom	Space	Room	Lot \
count	156.000000	156.000000	146.000000	156.000000	146.000000
mean	56.474359	3.166667	1097.246575	6.500000	32.808219
std	12.875307	1.348037	462.540698	1.675247	8.457859
min	32.000000	1.000000	539.000000	4.000000	24.000000
25%	46.000000	2.000000	805.250000	5.000000	25.000000
50%	55.000000	3.000000	965.500000	6.000000	30.000000
75%	65.000000	4.000000	1220.500000	7.000000	37.000000
max	90.000000	8.000000	2295.000000	12.000000	50.000000

```
                Tax    Bathroom      Garage   Condition
count   147.000000  156.000000  156.000000  156.000000
mean    911.707483    1.480769    0.846154    0.230769
std     443.263430    0.529408    0.808454    0.422682
min     418.000000    1.000000    0.000000    0.000000
25%     652.500000    1.000000    0.000000    0.000000
50%     821.000000    1.500000    1.000000    0.000000
75%    1012.500000    2.000000    1.500000    0.000000
max    2752.000000    3.000000    2.000000    1.000000
```

Realest data'sına küçük bir bakış attık. Her değişken float. Yani kategorik değişken yok. 157 satır, 9 sütundan oluşuyor. Condition değişkeni ise 1 ve 0 olarak verilmiş.

Verinin hikayesi kısaca, Chicago'daki evlerin fiyatlarına etki edebilecek değişkenleri gösteriyor. Evin oda sayısı, garaj sayısı, Evin durumu(iyi = 1, kötü = 0). Bu tarz değişkenlerle açıklamaya çalışan bir veri.

Bu veri üzerinde çeşitli regresyon modellerini kullanarak bu projeyi gerçekleştireceğim.

1.3 Eksik Veri Kontrolü

[8]:
```
data.isnull().sum().sum()
```

[8]:
```
np.int64(38)
```

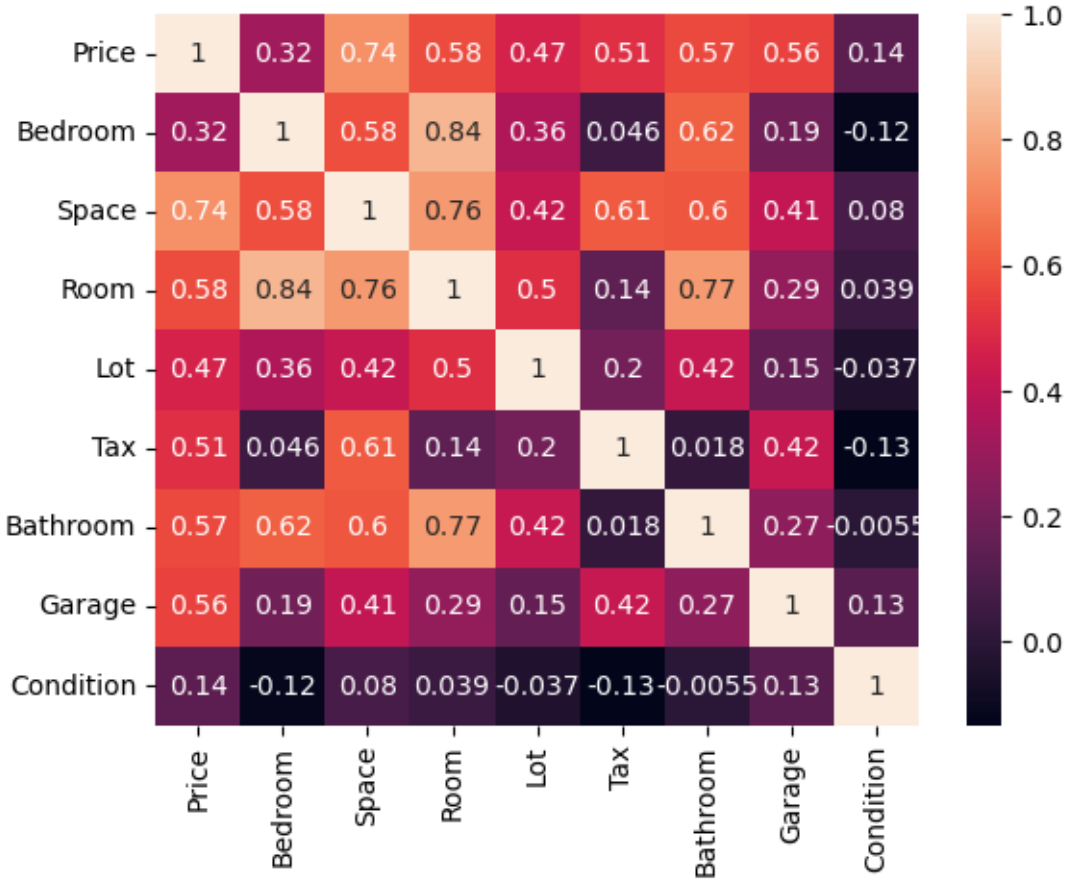
Toplam 38 adet eksik gözlemimiz var. Bunları silelim.

[14]:
```
data = data.dropna()
```

1.4 Korelasyon Kontrolü

[10]:
```
corr = data.corr()

mt.figure()
sb.heatmap(corr, annot=True)
mt.show()
```



Bağımsız değişkenlerin birbiriyle bağlantısı belli miktar var gibi gözüküyor fakat çok da yüksek değil. Fazla multicollinearity olmadığını aslında buradan görebiliriz. Şimdi yavaştan modellerimizi kurarak başlayalım. İlk olarak doğrusal modellerle sonra doğrusal olmayan modellerle ilerleyeceğiz.

2 LİNEER REGRESYON MODELLERİ

2.1 BASİT DOĞRUSAL REGRESYON

Basit doğrusal regresyon için bağımsız değişken olarak "Garage", bağımlı değişken de zaten "Price" olarak seçildi.

[11]:
```
x = data[["Garage"]]
y = data[["Price"]]
```

[12]:
```
x = sm.add_constant(x)
```

Açıklayıcı değişken setinin başına 1 kolonu koyduk çünkü statsmodel bunu otomatik yapmıyor.

[13]:
```
model_basit = sm.OLS(y,x).fit()
model_basit.summary()
```

[13]:

Dep. Variable:	Price	**R-squared:**	0.308
Model:	OLS	**Adj. R-squared:**	0.303
Method:	Least Squares	**F-statistic:**	56.16
Date:	Sat, 04 Apr 2026	**Prob (F-statistic):**	1.03e-11
Time:	13:46:36	**Log-Likelihood:**	-489.23
No. Observations:	128	**AIC:**	982.5
Df Residuals:	126	**BIC:**	988.2
Df Model:	1		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P> |t|**	**[0.025**	**0.975]**
const	49.2919	1.397	35.274	0.000	46.526	52.057
Garage	9.1407	1.220	7.494	0.000	6.727	11.555

Omnibus:	6.291	**Durbin-Watson:**	1.221
Prob(Omnibus):	0.043	**Jarque-Bera (JB):**	5.190
Skew:	0.396	**Prob(JB):**	0.0746
Kurtosis:	2.412	**Cond. No.**	2.46

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Modelimizi kurduk ve özetini inceleyelim.

- Düşük bir R^2 değerine sahip (0.303) çünkü sadece 1 tane açıklayıcı değişkeni var. Garaj değişkeninin %30 oranda fiyat değişkenindeki değişimi açıklayabildiğini gördük aslında.
- Anlamlı bir model.
- Garaj sayısının 1 adet artması evin fiyatını 9 birim artırmış.
- Normallikten hafif sapan bir model olmuş gibi gözüküyor. (JB olasılığı normal dağıldığını gösteriyor küçük bir farkla fakat Omnibus normal dağılmıyor diyor.)
- Kurtosis değeri de 3'e bi tık uzak kalmış(2.41). Yani normallikten biraz sapan bir model.
- 1.22 DW değeri ile de aslında otokorelasyon sorunu olan bir model gibi duruyor.
- 2.46 Cond.No. değeri de fazla multicollinearity olmadığını gösteriyor.

Şimdi modelin MSE ve R^2 değerlerine bakalım.

[14]:
```
mean_squared_error(y, model_basit.predict(x))
```

[14]: 122.29079341317366

[15]:
```
r2_score(y, model_basit.predict(x))
```

[15]: 0.308293945199157

122.29 MSE ve 0.30 R^2 değerine sahip olduğunu gördük.

2.2 ÇOKLU DOĞRUSAL REGRESYON

İlk olarak genel bir model kuralım sonra eğitim test parçalanması yaparız.

[16]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]
```

[17]:
```
x = sm.add_constant(x)
```

[18]:
```
model_coklu = sm.OLS(y,x).fit()
model_coklu.summary()
```

[18]:

Dep. Variable:	Price	**R-squared:**	0.718
Model:	OLS	**Adj. R-squared:**	0.699
Method:	Least Squares	**F-statistic:**	37.90
Date:	Sat, 04 Apr 2026	**Prob (F-statistic):**	2.76e-29
Time:	13:46:36	**Log-Likelihood:**	-431.77
No. Observations:	128	**AIC:**	881.5
Df Residuals:	119	**BIC:**	907.2
Df Model:	8		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P> |t|**	**[0.025**	**0.975]**
const	19.9935	4.081	4.899	0.000	11.912	28.075
Bedroom	-3.1602	0.931	-3.394	0.001	-5.004	-1.317
Space	0.0097	0.004	2.611	0.010	0.002	0.017
Room	1.6890	1.120	1.508	0.134	-0.529	3.907
Lot	0.2362	0.090	2.634	0.010	0.059	0.414
Tax	0.0044	0.003	1.490	0.139	-0.001	0.010
Bathroom	6.3758	2.018	3.160	0.002	2.380	10.371
Garage	4.1448	0.956	4.337	0.000	2.253	6.037
Condition	1.8358	1.797	1.021	0.309	-1.723	5.394

Omnibus:	0.338	**Durbin-Watson:**	1.900
Prob(Omnibus):	0.845	**Jarque-Bera (JB):**	0.361
Skew:	0.120	**Prob(JB):**	0.835
Kurtosis:	2.899	**Cond. No.**	1.01e+04

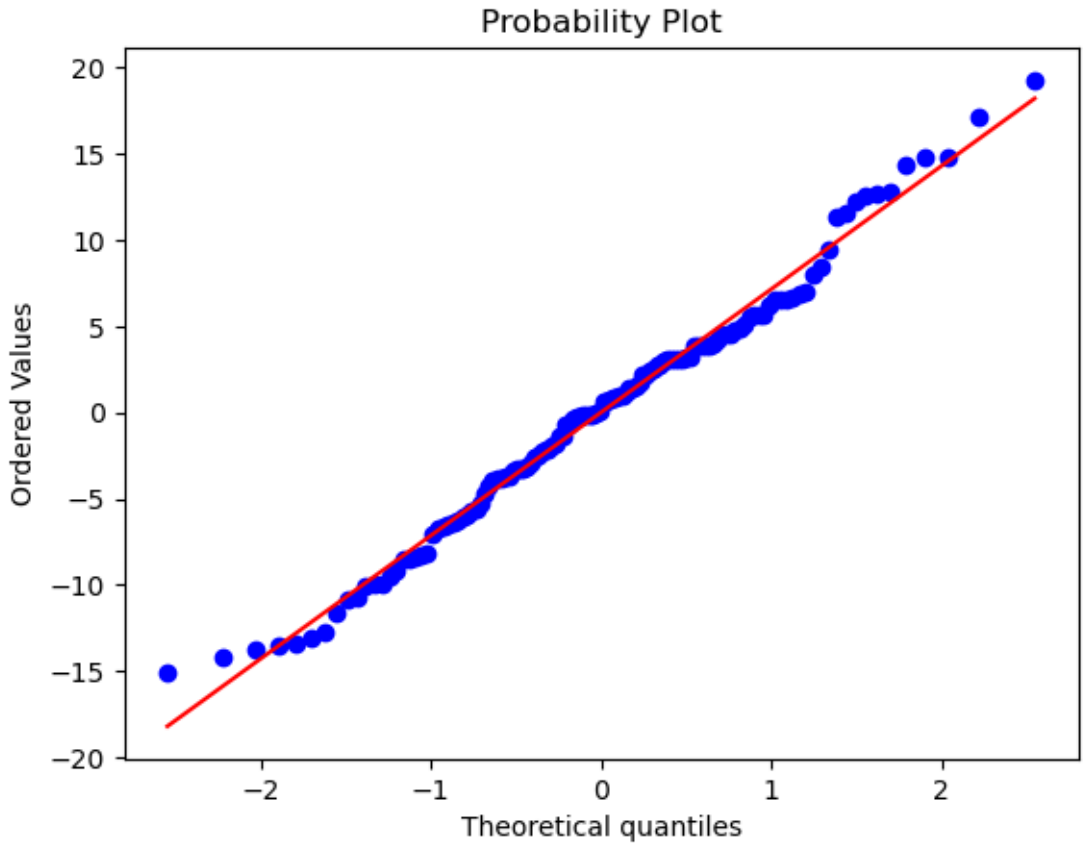
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.01e+04. This might indicate that there are
strong multicollinearity or other numerical problems.

Özetini inceleyelim; - Anlamlı bir model. - Bağımsız değişkenler %69.9 oranında fiyat değişkenini açıklayabilmiş. - Yatak odası sayısının 1 artması fiyatı 6.3 birim artırmış, garaj sayısının 1 artması fiyatı 4.1 birim artırmış. İlginç şekilde yatak odası sayısının 1 birim artması fiyat değişkenini 3 birim azaltmış. Diğer durumlar ise küçük etkiler göstermiş bu değişkenlere göre. - Normal dağılıma uyan bir model. JB ve Omnibus değerleri bunu gösteriyor. Aynı zamanda Skew 0'a çok yakın ve Kurtosis değeri de 3'e yakın. - DW değeri ise otokorelasyon olmadığını gösteriyor. - Cond.No. değeri de 0'a çok yakın yani multicollinearity yok diyebiliriz. - Ortalama iyilikte bir model olduğunu söyleriz fakat bundan daha iyileri de olacaktır.

2.3 Varsayım Kontrollerini Grafikle İnceleyelim

İlk olarak normalliğe bakalım;

[19]:
```
residuals = model_coklu.resid
stats.probplot(residuals, dist ="norm", plot = pylab)
mt.show()
```



Görüldüğü üzere normallikten pek sapma yok gibi burada da.

Test ile de bakalım;

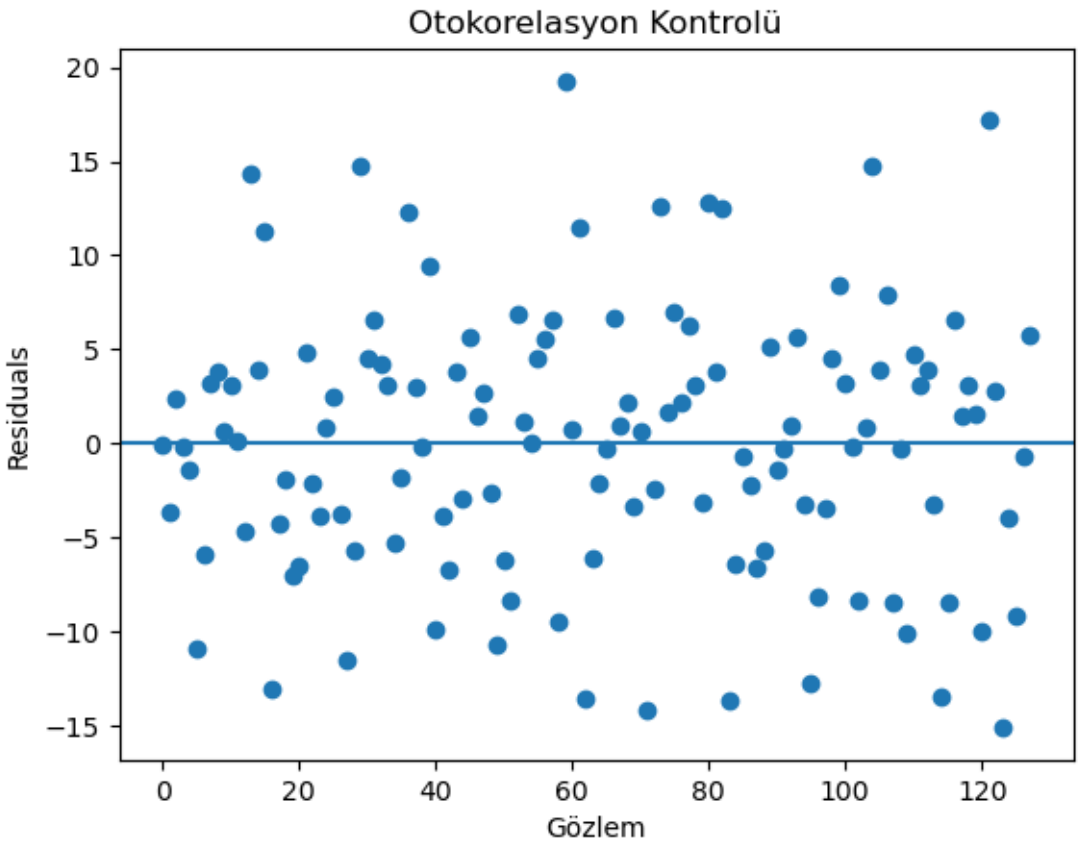
[20]:
```
shapiro(residuals)
```

[20]:
```
ShapiroResult(statistic=np.float64(0.9877829624394414),
pvalue=np.float64(0.3132658483535822))
```

H0 reddedilemez yani normal dağılan bir model.

Otokorelasyona da grafikle bakalım;

[21]:
```
mt.scatter(range(len(residuals)), residuals)
mt.title("Otokorelasyon Kontrolü")
mt.xlabel("Gözlem")
mt.ylabel("Residuals")
mt.axhline(y=0)
mt.show()
```

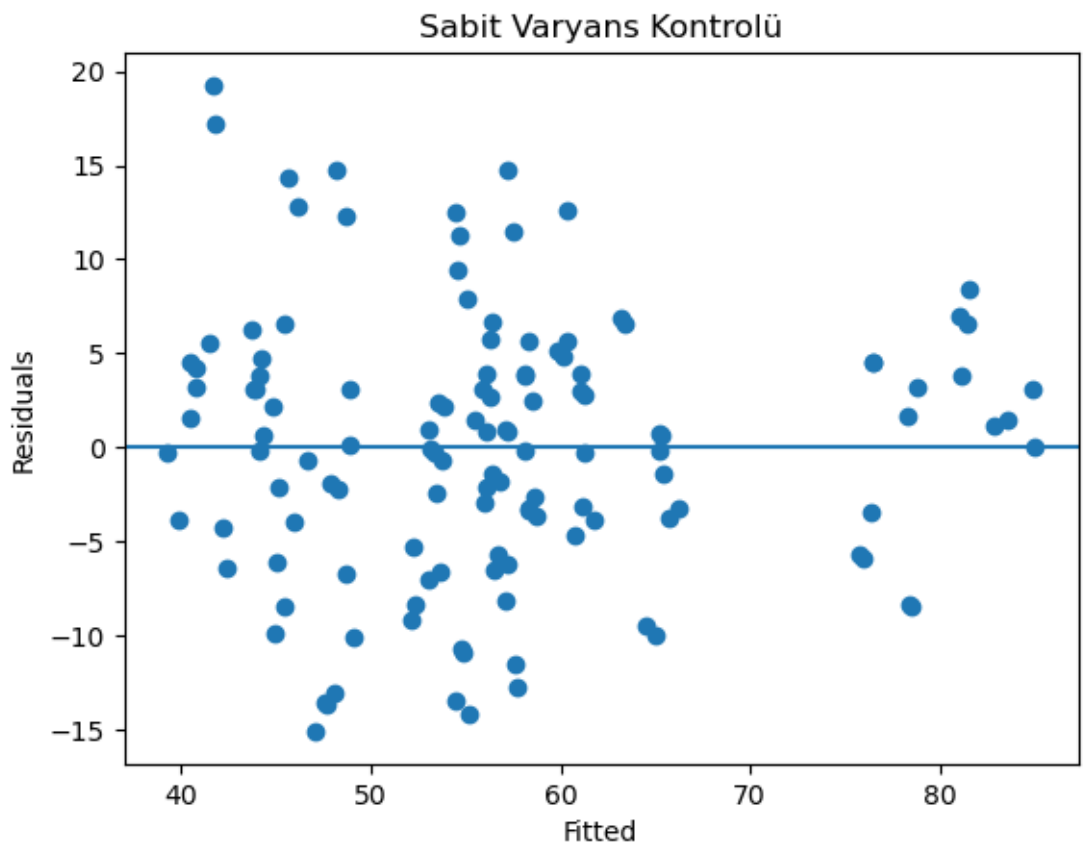


Tamamen rastgele bir dağılım var yani otokorelasyon yok.

Değişen varyans sorunu var mı diye de bakalım. Summary'de gözüken bir değer değil çünkü.

[22]:
```
mt.scatter(model_coklu.fittedvalues, residuals)
mt.ylabel("Residuals")
mt.xlabel("Fitted")
mt.title("Sabit Varyans Kontrolü")
mt.axhline(y=0)
```

[22]: <matplotlib.lines.Line2D at 0x191662fe5d0>



Kapanan bir huni gibi gözüküyor buradan modelde değişen varyans sorunu var deriz. Bunu düzeltmek için veriye dönüşüm yapmak gerekiyor fakat bu projede bununla uğraşmayacağız.

Multicollinearity'ye grafikle bakmaya gerek yok. Zaten korelasyon matrisi ve koşul indeksi kullanarak görmüştük fazla multicollinearity olmadığını.

Şimdi çoklu doğrusal regresyon modelimizin de MSE ve R^2 değerlerini inceleyelim;

[23]:
```
mean_squared_error(y, model_coklu.predict(x))
```

[23]: 49.82715429623934

[24]:
```
r2_score(y, model_coklu.predict(x))
```

[24]: 0.7181656659650734

49.827 MSE ve 0.718 R^2 değeri ile basit doğrusal regresyon modelinden çok daha iyi bir modelle karşılaştık.

2.4 Eğitim Test Parçalanması

[25]:
```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
```

[26]:
```
x_train.shape
```

[26]:
```
(102, 9)
```

102 satır eğitim setinde oldu.

[27]:
```
x_test.shape
```

[27]:
```
(26, 9)
```

26 satır ise test setinde.

Statsmodel kütüphanesi ile eğitim modelimizi kuralım;

[28]:
```
model_egitim = sm.OLS(y_train,x_train).fit()
model_egitim.summary()
```

[28]:

Dep. Variable:	Price	**R-squared:**	0.716
Model:	OLS	**Adj. R-squared:**	0.691
Method:	Least Squares	**F-statistic:**	29.26
Date:	Sat, 04 Apr 2026	**Prob (F-statistic):**	2.86e-22
Time:	13:46:38	**Log-Likelihood:**	-343.40
No. Observations:	102	**AIC:**	704.8
Df Residuals:	93	**BIC:**	728.4
Df Model:	8		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P> |t|**	**[0.025**	**0.975]**
const	21.7686	4.790	4.545	0.000	12.257	31.281
Bedroom	-2.2925	1.051	-2.181	0.032	-4.379	-0.206
Space	0.0101	0.004	2.479	0.015	0.002	0.018
Room	0.5239	1.311	0.400	0.690	-2.079	3.127
Lot	0.2767	0.101	2.750	0.007	0.077	0.476
Tax	0.0034	0.003	1.040	0.301	-0.003	0.010
Bathroom	7.2912	2.404	3.033	0.003	2.517	12.066
Garage	5.1164	1.079	4.742	0.000	2.974	7.259
Condition	1.8999	2.076	0.915	0.362	-2.222	6.022

Omnibus:	0.833	**Durbin-Watson:**	2.193
Prob(Omnibus):	0.659	**Jarque-Bera (JB):**	0.543
Skew:	0.173	**Prob(JB):**	0.762
Kurtosis:	3.092	**Cond. No.**	1.07e+04

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.07e+04. This might indicate that there are strong multicollinearity or other numerical problems.

Bunu da ekstra incelemeye gerek yok ana modelimiz gibi varsayımları sağlıyor ve anlamlı bir model.

Şimdi test seti üzeinden tahmin oluşturup buradan MSE ve R^2 değerlerini kontrol edelim;

[29]:
```
y_tahmin = model_egitim.predict(x_test)
```

[30]:
```
mean_squared_error(y_test, y_tahmin)
```

[30]: 57.58007022465131

[31]:
```
r2_score(y_test, y_tahmin)
```

[31]: 0.6994740003716469

57.58 MSE ve 0.6994 R^2 değeri geldi. Ana modele göre biraz daha kötü geldi fakat zaten bütün modelleri test setinde kıyaslayacağımız için önemli olan burada çıkan sonuç. Bu bizim ilk test seti değerlerimiz.

MAPE'yi de kontrol edelim;

[32]:
```
mean_absolute_percentage_error(y_test, y_tahmin)
```

[32]: 0.1214949154762556

Model %12.1 hatalı olacak şekilde tahmin etmiş.

2.5 PCR (PRINCIPAL COMPONENT REGRESSION)

İlk olarak "KFold Cross Validation" yapısını belirleyelim;

[33]:
```
cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

Modelimizin x ve y'sini tekrar yazalım ve eğitim test parçalanmasını yapalım;

[34]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
```

PCA ile boyut indirgemesi yapalım; (scale ederek).

[35]:
```
pca = PCA()
x_train_scaled = pca.fit_transform(scale(x_train))
x_test_scaled = pca.transform(scale(x_test))
```

Şimdi optimal component sayısını belirleyelim. 8 adet bağımsız değişkenimiz var bu nedenle döngüyü ona göre kurduk.

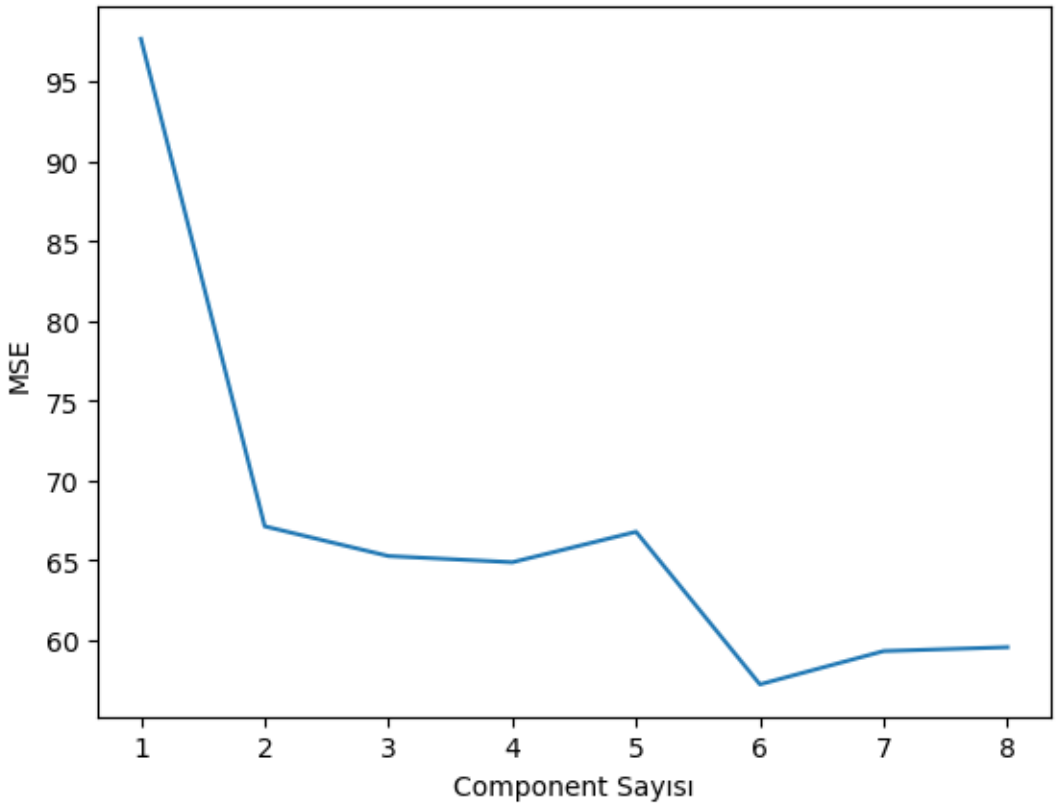
[36]:
```
regr = LinearRegression()
mse = []

for i in np.arange(1,9):
    score = -1*cross_val_score(regr, x_train_scaled[:,:i], y_train, cv = cv,
                                                    scoring =␣
 ↪'neg_mean_squared_error').mean()
    mse.append(score)
```

Grafik ile bakalım hangi component en iyi MSE'yi vermiş.

[37]:
```
mt.plot(range(1, 9), mse)
mt.xlabel("Component Sayısı")
mt.ylabel("MSE")
```

[37]: Text(0, 0.5, 'MSE')



6 componentin en iyi olduğunu belirledik. Şimdi bu component sayısı ile modeli tekrar fit edelim. Ve tekrar scale edelim;

[38]:
```
pca = PCA(n_components= 6)
x_train_pca = pca.fit_transform(scale(x_train))
```

```
x_test_pca = pca.transform(scale(x_test))

regr = LinearRegression()
pcreg = regr.fit(x_train_pca, y_train)
```

Şimdi MSE ve R^2 değerlerini kontrol edelim;

[39]:
```
y_test_pred = pcreg.predict(x_test_pca)
```

[40]:
```
mean_squared_error(y_test, y_test_pred)
```

[40]:
```
56.053605610761615
```

[41]:
```
r2_score(y_test, y_test_pred)
```

[41]:
```
0.7074410331001015
```

56.053 MSE ve 0.707 R^2 değeri geldi. Çoklu doğrusal regresyon modeline göre küçük bir farkla daha iyi geldi yani.

[42]:
```
mean_absolute_percentage_error(y_test, y_test_pred)
```

[42]:
```
0.1206873548595223
```

Model %12.06 hatalı olacak şekilde tahmin etmiş. Yani çoklu doğrusal regresyon modelinden yine daha iyi ama küçük bir farkla.

2.6 PLSR (PARTIAL LEAST SQUARES REGRESSION)

[43]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)

cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

[44]:
```
pls_model = PLSRegression().fit(x_train, y_train)
pls_model.coef_
```

[44]:
```
array([[-0.65452443,  0.00739609,  0.80638196,  0.22353776,  0.00689395,
         4.1848117 ,  4.86932956,  3.87278039]])
```

En uygun componenti belirlemeden önce ana modelimizi kurduk ve katsayılarını inceledik.

Şimdi en uygun component sayısını belirleyelim;

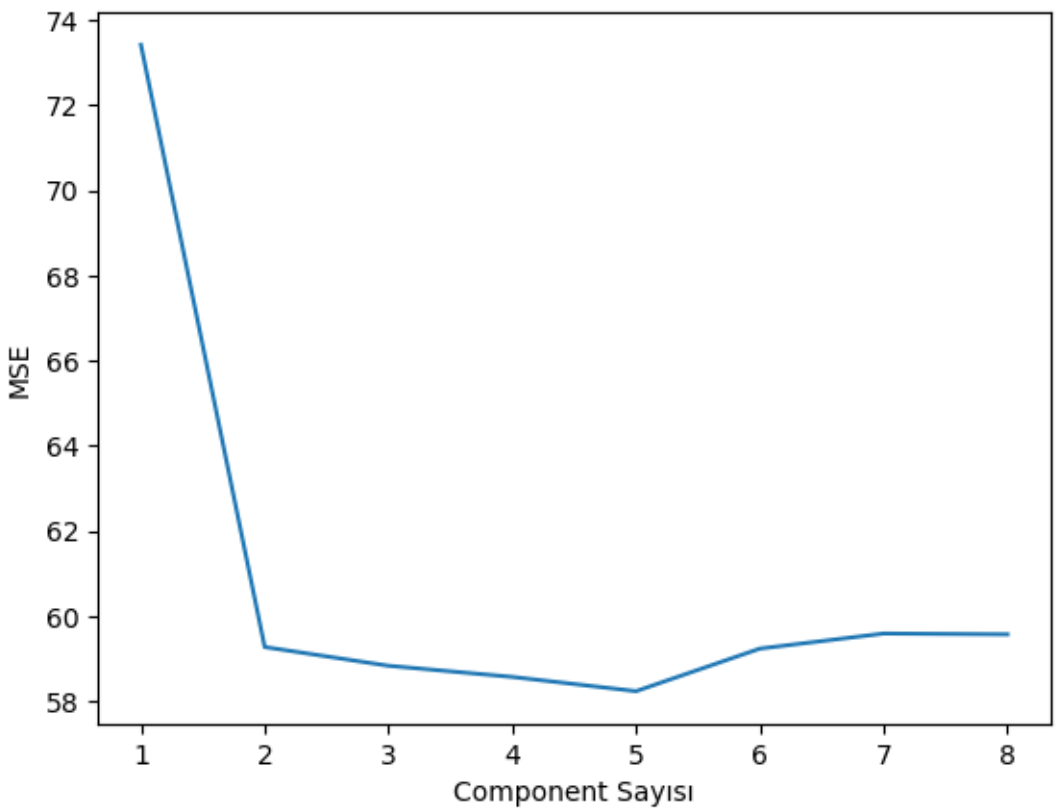
[45]:
```
mse = []
for i in range(1, 9):
    pls = PLSRegression(n_components=i)
```

```
    score = -1 *cross_val_score(pls, x_train, y_train, cv=cv,
                                scoring='neg_mean_squared_error').mean()
    mse.append(score)
```

Grafik ile kontrol edelim;

[46]:
```
mt.plot(range(1, 9), mse)
mt.xlabel("Component Sayısı")
mt.ylabel("MSE")
```

[46]: Text(0, 0.5, 'MSE')



En uygun component sayısının 5 olduğunu gördük. Bununla tekrar modeli fit edelim.

[47]:
```
pls_model = PLSRegression(n_components=5).fit(x_train, y_train)
```

Tahminleri oluşturup MSE ve R^2 değerlerini inceleyelim;

[48]:
```
y_test_predict = pls_model.predict(x_test)
```

[49]:
```
mean_squared_error(y_test, y_test_predict)
```

[49]: 57.84541436405782

[50]:
```
r2_score(y_test, y_test_predict)
```

[50]: 0.6980890973586853

57.845 MSE ve 0.698 R^2 değeri çıktı. Yani hala PCR daha iyi bir model fakat birbirlerine çok yakınlar.

[51]:
```
mean_absolute_percentage_error(y_test, y_test_predict)
```

[51]: 0.12101304110852348

Model %12.1 hatalı olacak şekilde tahmin etmiş. Bu değer ise yine küçük bir farkla PCR'den kötü gelmiş.

2.7 RIDGE REGRESYONU

[52]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)

cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

İlk olarak lambda parametre optimizasyonu yapıp ona göre Ridge modelini kuralım.

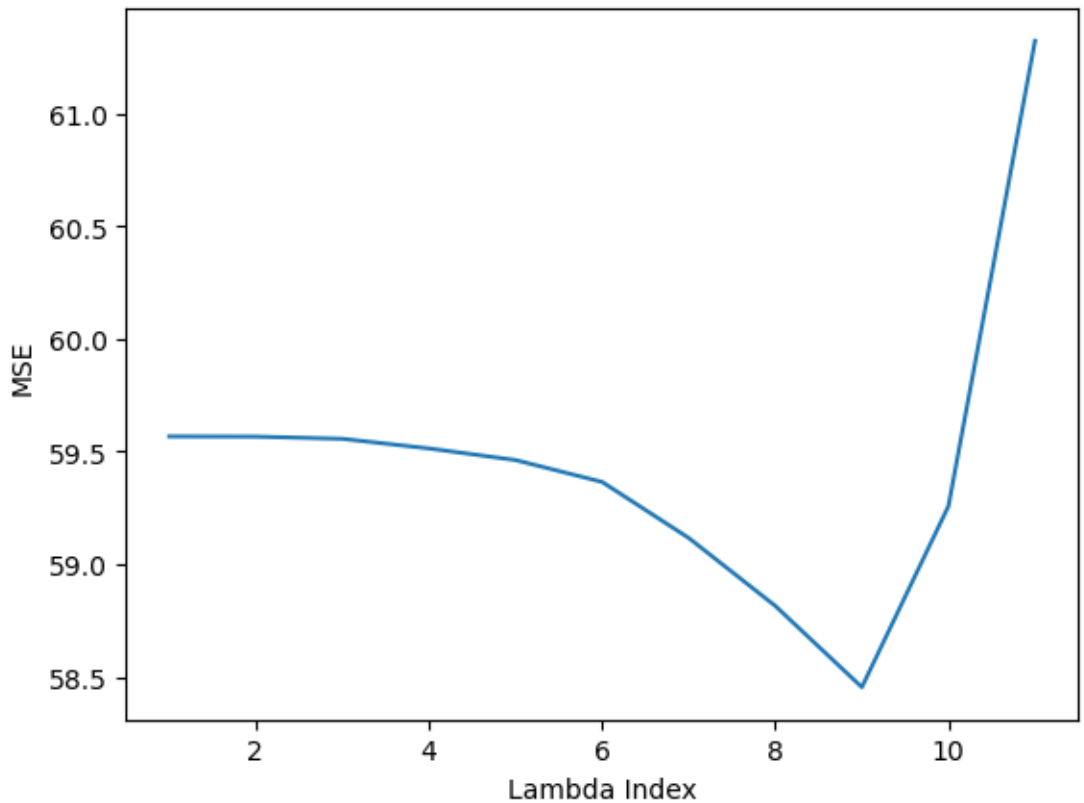
[53]:
```
r2 = []
mse_sonuc = []
alpha_cand = np.array([0.0001, 0.001, 0.01, 0.05, 0.1, 0.2, 0.5, 1, 5, 10, 20])
```

[54]:
```
for i in alpha_cand:
    model_i = Ridge(alpha=i)
    mse = -1 * cross_val_score(model_i, x_train, y_train,
                               cv=cv,
                               scoring='neg_mean_squared_error').mean()
    r2_cv = cross_val_score(model_i, x_train, y_train,
                            cv=cv,
                            scoring='r2').mean()
    mse_sonuc.append(mse)
    r2.append(r2_cv)
```

Grafikle inceleyelim;

[55]:
```
mt.plot(range(1, 12), mse_sonuc)
mt.xlabel("Lambda Index")
mt.ylabel("MSE")
```

[55]: Text(0, 0.5, 'MSE')



9. index değeri en iyi MSE geldi bu da 5 değerine eşit oluyor. Yani en iyi alpha değerimiz 5 oldu.

[56]:
```
r2
```

[56]:
```
[np.float64(0.6231059633222301),
 np.float64(0.6231140495850835),
 np.float64(0.6231945988520966),
 np.float64(0.6235458006733534),
 np.float64(0.623969638661064),
 np.float64(0.6247694174423934),
 np.float64(0.6268299686176503),
 np.float64(0.6293707526068486),
 np.float64(0.6334949304404892),
 np.float64(0.6288465752193029),
 np.float64(0.616034370080315)]
```

En büyük R^2 değeri de aynıu şekilde gelmiş yani kesinlikle 5 değeri optimum alpha.

Bu alpha değeri ile final modelimizi kuralım;

[57]:
```
final_model = Ridge(alpha = 5).fit(x_train,y_train)
```

Tahminleri oluşturup MSE ve R^2 değerlerini kontrol edelim;

[58]:
```
y_test_predict = final_model.predict(x_test)
```

[59]:
```
mean_squared_error(y_test, y_test_predict)
```

[59]: 54.991972323670424

[60]:
```
r2_score(y_test, y_test_predict)
```

[60]: 0.7129819850926404

54.99 MSE ve 0.712 R^2 değeri geldi. Yani şu ana kadar elde ettiğimiz en iyi model bu oldu. Küçük bir farkla olsa da.

[61]:
```
mean_absolute_percentage_error(y_test, y_test_predict)
```

[61]: 0.11880118230281687

Model %11.88 hatalı olacak şekilde tahmin etmiş. Yani yine küçük bir farkla şu ana kadarki en iyi modelimiz bu.

2.8 LASSO REGRESYONU

[62]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)

cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

İlk olarak parametre optimizasyonu ile başlayalım;

[63]:
```
y_train = y_train.squeeze()
lasso_model = LassoCV(cv = cv, random_state = 42).fit(x_train, y_train)
lasso_model.alpha_
```

[63]: np.float64(7.0316231955797655)

7.031 alpha değerini optimum olarak belirledik. Burada squeeze kullanmamın sebebi Python'ın y_train için boyut hatası vermesi. Bu hatayı bu şekilde düzelttim. Fakat çıkan sonuçta herhangi bir değişikliğe sebep olacak bir şey değil.

Şimdi bu alpha değerini kullanarak final modelini kuralım;

[64]:
```
final_model = Lasso(alpha = lasso_model.alpha_).fit(x_train, y_train)
```

Tahminleri oluşturalım ve MSE, R^2 değerlerini inceleyelim;

[65]:
```
y_test_predict = final_model.predict(x_test)
```

[66]:
```
mean_squared_error(y_test, y_test_predict)
```

[66]: 66.25605858012275

[67]:
```
r2_score(y_test, y_test_predict)
```

[67]: 0.6541916646065242

66.256 MSE ve 0.654 R^2 değerleri ile diğer modellerimizden kötü bir sonuç aldık fakat yine de ortalama bir model diyebiliriz.

[68]:
```
mean_absolute_percentage_error(y_test, y_test_predict)
```

[68]: 0.12912226730777043

Model %12.91 hatalı olacak şekilde tahmin etmiş. Yani gerçekten en kötü model bu oldu tabi basit doğrusal regresyon modelini dışarıda tutarak.

3 LİNEER OLMAYAN REGRESYON MODELLERİ

3.1 KNN REGRESYONU

[69]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

İlk olarak GridSearch kullanarak optimum k sayısını belirleyelim.

[70]:
```
KNN_arg = KNeighborsRegressor()
k_params = {"n_neighbors":[1,2,3,4,5,6,7,8,9,10,20,30,40]}
grid = GridSearchCV(KNN_arg,k_params,cv=cv).fit(x_train,y_train)
grid.best_params_
```

[70]: {'n_neighbors': 2}

Optimum k değeri 2 geldi. Şimdi bu değeri kullanarak final modelini kuralım;

[71]:
```
final_model = KNeighborsRegressor(n_neighbors = 2).fit(x_train, y_train)
```

Tahminleri oluşturup MSE ve R^2 değerlerini kontrol edelim;

[72]:
```
y_test_predict = final_model.predict(x_test)
```

[73]:
```
mean_squared_error(y_test, y_test_predict)
```

[73]: 17.576923076923077

[74]:
```
r2_score(y_test, y_test_predict)
```

[74]: 0.9082612723903645

17.576 MSE ve 0.9082 R^2 değeri ile şu ana kadarki en iyi model KNN oldu. Bu aslında demek oluyor ki verimizde lineer olmayan ilişkiler mevcut ve KNN bunu çok iyi yakaladı.

[75]:
```
mean_absolute_percentage_error(y_test, y_test_predict)
```

[75]: 0.06701323339836086

Model %6.70 hatalı olacak şekilde tahmin etmiş. Yani gerçekten büyük bir farkla en iyi model oldu.

3.2 SVR (SUPPORT VECTOR MACHINE) REGRESYONU (DOĞRUSAL)

[76]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

İlk olarak StandardScaler ile Scale edelim;

[77]:
```
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Parametre optimizasyonu yapalım;

[78]:
```
y_train = y_train.squeeze()
params_svr = {"C": np.arange(0.1,4,0.1)}
svr_m = SVR(kernel = "linear")
SR_model_cv = GridSearchCV(svr_m,params_svr,cv = cv)
SVR_model_fit = SR_model_cv.fit(x_train_scaled,y_train)

SVR_model_fit.best_params_
```

[78]: {'C': np.float64(0.30000000000000004)}

Yine squeeze kullandık çıktıda verilen sorunu düzeltmek için. GridSearch ile de en iyi parametreyi bulduk. O da 0.3 oldu.

Şimdi bunu kullanarak final modeli kuralım;

[79]:
```
SVR_model = SVR(kernel = "linear", C = 0.3).fit(x_train_scaled,y_train)
```

Tahminleri oluşturup MSE ve R^2 değerlerine bakalım;

[80]:
```
y_test_predict = SVR_model.predict(x_test_scaled)
```

[81]:
```
mean_squared_error(y_test,y_test_predict)
```

[81]: 65.18763521349173

[82]:
```
r2_score(y_test,y_test_predict)
```

[82]: 0.6597680558653458

65.187 MSE ve 0.659 R^2 değeri geldi. Yine ortalama bir model olduğunu görüyoruz. Zaten bu model doğrusal olduğundan dolayı KNN kadar başarılı ilişki yakalayamadı çünkü verimizde doğrusal olmayan ilişkiler olduğunu biliyoruz.

[83]:
```
mean_absolute_percentage_error(y_test, y_test_predict)
```

[83]: 0.12241545511494278

Model %12.24 hatalı olacak şekilde tahmin etmiş. Yani diğer lineer modeller gibi bir sonuç vermiş.

3.3 SVR (SUPPORT VECTOR MACHINE) REGRESYONU (DOĞRUSAL OLMAYAN)

[84]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

Scale edelim;

[85]:
```
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Parametre optimizasyonu;

[86]:
```
y_train = y_train.squeeze()
params_svr = {"C":np.arange(0.1,4,0.1)}
svr_m = SVR(kernel = "rbf")
SR_model_cv = GridSearchCV(svr_m,params_svr,cv = cv)
SVR_model_fit = SR_model_cv.fit(x_train_scaled,y_train)
SVR_model_fit.best_params_
```

[86]: {'C': np.float64(3.9000000000000004)}

En iyi 3.9 değeri gelmiş oldu. Final modeli buna göre tekrar kuralım;

[87]:
```
SVR_model = SVR(kernel = "rbf", C = 3.9).fit(x_train_scaled,y_train)
```

Tahminleri oluşturup MSE ve R^2 değerlerine bakalım;

[88]:
```
y_test_predict = SVR_model.predict(x_test_scaled)
```

[89]:
```
mean_squared_error(y_test,y_test_predict)
```

[89]: 44.38889969790718

[90]:
```
r2_score(y_test,y_test_predict)
```

[90]: 0.7683222962030168

44.388 MSE ve 0.768 R^2 değeri ile lineer modellerden iyi bir model olduğunu görüyoruz fakat KNN çok daha iyi.

[91]:
```
mean_absolute_percentage_error(y_test, y_test_predict)
```

[91]: 0.11008470730763853

Model %11.008 hatalı olacak şekilde tahmin etmiş. Yani ortalama diyebiliriz.

3.4 YAPAY SİNİR AĞLARI (ANN) (MLP)

[92]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

Scale edelim;

[93]:
```
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Ana modeli kuralım ardından da parametre optimizasyonu yapalım;

[94]:
```
y_train = y_train.squeeze()
ann_model = MLPRegressor(solver = 'lbfgs',
                         random_state = 42,
                         learning_rate_init = 0.02,
                         max_iter = 5000).fit(x_train_scaled,y_train)
```

[95]:
```
ann_params = {"alpha":[0.01,0.05,0.1],
              "hidden_layer_sizes":[(50,),(100,),(300,200,100)],
              "activation":["relu"]}

ann_grid_search = GridSearchCV(ann_model,
                               ann_params,
                               n_jobs = -1,
                               cv = cv).fit(x_train_scaled,y_train)
```

```
ann_grid_search.best_params_
```

[95]: {'activation': 'relu', 'alpha': 0.1, 'hidden_layer_sizes': (100,)}

En iyi parametreleri bulduk. Şimdi final modeli kuralım;

[96]:
```
ann_model = MLPRegressor(activation = "relu",
                         alpha = 0.05,
                         random_state=42,
                         hidden_layer_sizes= (300,200,100),
                         max_iter = 5000).fit(x_train_scaled,y_train)
```

Tahminleri oluşturalım ve MSE ile R^2 değerlerine bakalım;

[97]:
```
ann_preds = ann_model.predict(x_test_scaled)
```

[98]:
```
mean_squared_error(y_test, ann_preds)
```

[98]: 16.33513401275621

[99]:
```
r2_score(y_test,ann_preds)
```

[99]: 0.9147425062336072

16.335 MSE ve 0.914 R^2 değeri geldi. KNN'den daha iyi bir modelle karşılaşmış olduk. Şu ana kadarki en iyi model.

[100]:
```
mean_absolute_percentage_error(y_test, ann_preds)
```

[100]: 0.05788476573508586

Model %5,78 hatalı olacak şekilde tahmin etmiş. Yani yine şu ana kadarki en iyisi.

4 CART - (CLASSIFICATION AND REGRESSION TREES)

4.1 Decision Tree

[101]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

Ana modeli kuralım;

[102]:
```
cart_model = DecisionTreeRegressor(random_state = 42).fit(x_train, y_train)
```

Parametre optimizasyonu yapalım;

[103]:
```
cart_params = {
    "min_samples_split": range(2, 100),
    "max_leaf_nodes": range(2, 10)}
grid_cart_model = GridSearchCV(cart_model, cart_params, cv = cv).fit(x_train,␣
 ↪y_train)
grid_cart_model.best_params_
```

[103]:
```
{'max_leaf_nodes': 9, 'min_samples_split': 9}
```

En iyi parametreleri bulduk, şimdi final modeli kuralım;

[104]:
```
optim_cart_model = DecisionTreeRegressor(max_leaf_nodes = 9,
                                          min_samples_split = 9,
                                         random_state = 42).fit(x_train, y_train)
```

Tahminleri oluşturalım;

[105]:
```
best_pred_cart = optim_cart_model.predict(x_test)
```

[106]:
```
mean_squared_error(y_test, best_pred_cart)
```

[106]:
```
59.07436423427335
```

[107]:
```
r2_score(y_test, best_pred_cart)
```

[107]:
```
0.6916748747500866
```

59.074 MSE ve 0.691 R^2 ile ortalama bir sonuç geldi lineer modellerden pek farkı olmadı.

[108]:
```
mean_absolute_percentage_error(y_test, best_pred_cart)
```

[108]:
```
0.11057670017994456
```

Model %11.05 hatalı olacak şekilde tahmin etmiş. Yani ortalama bir model.

4.2 BOOSTSTRAP AGGRETION (BAGGING)

[109]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

Ana modeli kurarak başlayalım;

[110]:
```
y_train = y_train.squeeze()
bagging_model = BaggingRegressor(bootstrap_features = True,
                                 random_state = 42).fit(x_train, y_train)
```

Parametre optimizasyonu yapalım;

[111]:
```
bag_params = {"n_estimators": range(2,40)}
grid_bag_model = GridSearchCV(bagging_model, bag_params, cv = cv).fit(x_train,␣
↪y_train)

grid_bag_model.best_params_
```

[111]:
```
{'n_estimators': 33}
```

Final modeli kuralım buna göre;

[112]:
```
best_bag_model = BaggingRegressor(bootstrap_features = True,
                                  random_state=42,
                                  n_estimators = 33).fit(x_train, y_train)
```

Tahminleri oluşturalım;

[113]:
```
best_bag_preds = best_bag_model.predict(x_test)
```

[114]:
```
mean_squared_error(y_test, best_bag_preds)
```

[114]:
```
14.126554641446408
```

[115]:
```
r2_score(y_test, best_bag_preds)
```

[115]:
```
0.9262696808398875
```

14.126 MSE ve 0.926 R^2 değeri ile elde ettiğimiz en iyi model bu model oldu.

[116]:
```
mean_absolute_percentage_error(y_test, best_bag_preds)
```

[116]:
```
0.06156939495473889
```

Model %6.15 hatalı olacak şekilde tahmin etmiş. Yani fazlasıyla iyi bir model.

4.3 RANDOM FOREST (RF)

[117]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

Scale edelim;

[118]:
```
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Ana modeli kuralım;

[119]:
```
y_train = y_train.squeeze()
rf_model = RandomForestRegressor(random_state = 42).fit(x_train_scaled, y_train)
```

Parametre optimizasyonu;

[120]:
```
rf_params = {"max_depth": range(1,10),
             "max_features":[2,3,5], "n_estimators":[100,200,300]}

rf_grid_model = GridSearchCV(rf_model, rf_params, cv = cv).fit(x_train_scaled,␣
 ↪y_train)

rf_grid_model.best_params_
```

[120]:
```
{'max_depth': 8, 'max_features': 3, 'n_estimators': 100}
```

Şimdi final modeli kuralım;

[121]:
```
rf_best_model = RandomForestRegressor(max_depth = 8,
                                      max_features = 3,
                                      n_estimators = 100,
                                      random_state = 42).fit(x_train_scaled,␣
 ↪y_train)
```

Tahminleri oluşturalım;

[122]:
```
pred_rf_best = rf_best_model.predict(x_test_scaled)
```

[123]:
```
mean_squared_error(y_test, pred_rf_best)
```

[123]:
```
12.254860582405184
```

[124]:
```
r2_score(y_test, pred_rf_best)
```

[124]:
```
0.9360385596532897
```

12.254 MSE ve 0.936 R^2 değeri ile en iyi model random forest modeli oldu.

[125]:
```
mean_absolute_percentage_error(y_test, pred_rf_best)
```

[125]:
```
0.05822498440456759
```

Model %5.82 hatalı olacak şekilde tahmin etmiş. Yani yine fazlasıyla iyi bir model.

4.4 GRADIENT BOOSTING

[15]:
```
x = data.drop("Price",axis = 1)
y = data[["Price"]]

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
```

```
cv = KFold(n_splits = 5, shuffle = True, random_state = 145)
```

Ana modeli kuralım;

[16]:
```
y_train = y_train.squeeze()
gb_model = GradientBoostingRegressor(random_state = 42).fit(x_train, y_train)
```

Parametre optimizasyonu yapalım;

[128]:
```
gb_params = {"learning_rate":[0.01, 0.1],
             "max_depth":[3,5,10],
             "n_estimators":[100, 200], "subsample":[1,0.5]}

gb_grid = GridSearchCV(gb_model, gb_params, cv = cv).fit(x_train, y_train)

gb_grid.best_params_
```

[128]:
```
{'learning_rate': 0.1, 'max_depth': 10, 'n_estimators': 100, 'subsample': 0.5}
```

Final modeli kuralım;

[17]:
```
gb_best_model = GradientBoostingRegressor(learning_rate = 0.1,
                                          max_depth = 10,
                                          n_estimators = 100,
                                          random_state=42,
                                          subsample = 0.5).fit(x_train, y_train)
```

Tahminleri oluşturalım;

[18]:
```
preds_gb_best = gb_best_model.predict(x_test)
```

[19]:
```
mean_squared_error(y_test, preds_gb_best)
```

[19]:
```
8.311899405267912
```

[20]:
```
r2_score(y_test, preds_gb_best)
```

[20]:
```
0.9566179431905412
```

8.311 MSE ve 0.956 R^2 ile en iyi model bu oldu.

[133]:
```
mean_absolute_percentage_error(y_test, preds_gb_best)
```

[133]:
```
0.047046946098362845
```

Model %4.70 hatalı olacak şekilde tahmin etmiş. Yani en iyi tahmin yapan modelimiz bu oldu.

5 TABLO İLE ÖZETLEYELİM

[134]:
```
from IPython.display import Image
```

[135]:
```
Image(filename="C:/Users/asus/Pictures/Screenshots/Tablo.png")
```

[135]:

Modeller	MSE	R^2	Sonuç
Basit Doğrusal Regresyon	122.290	0.308	Açıklayıcı değişken yetersizliğinden dolayı kötü ve yetersiz bir model.
Çoklu Doğrusal Regresyon	49.827	0.718	Ortalama sonuca ulaşılmış lineer bir model.
Principal Component Regression (PCR)	56.053	0.707	PCR yine ortalama bir sonuç vermiş. Çünkü zaten veride multicollinearity sorunu pek fazla yoktu.
Partial Least Squares Regression (PLSR)	57.845	0.698	Yine pek farklı sonuç çıkmamış ortalama bir model
Ridge Regresyonu	54.991	0.712	L2 cezalandırması yapılmış fakat sonuç yine ortalama kalmış.
Lasso Regresyonu	66.256	0.654	L1 cezalandırması yapılmış fakat bazı değişkenlerin katsayılarının sıfırlanması modeli daha kötü yapmış.
KNN Regresyonu	17.576	0.908	Lineer olmayan komşuluk temelli model çok daha iyi sonuç vermiş.
Support Vector Machine (Doğrusal)	65.187	0.659	Lineer olan bir model ve yine diğer lineer modeller gibi kötü sonuç vermiş.
Support Vector Machine (Doğrusal Olmayan)	44.388	0.768	Lineer olmayan SVR daha iyi fakat yine ortalama kalmış.
Yapay Sinir Ağları (ANN)(MLP)	16.335	0.914	KNN'den daha iyi bir sonuç gelmiş, gayet iyi bir model.
Decision Tree	59.074	0.691	Karar ağacı lineer modeller gibi bir sonuç vermiş demek ki veriye uymadı.
Booststrap Aggretion	14.126	0.926	Ensemble yöntemi gerçekten iyi bir sonuç vermiş.
Random Forest	12.254	0.936	Karar ağaçlarının fazlalaştırılması çok daha iyi bir model olmasını sağlamış.
Gradient Boosting	8.311	0.956	Bu ensemble yöntemi en iyi yöntem oldu. Hataları sıra sıra düzelterek ilerlemek en iyi modeli vermiş.

5.1 Genel Yorum

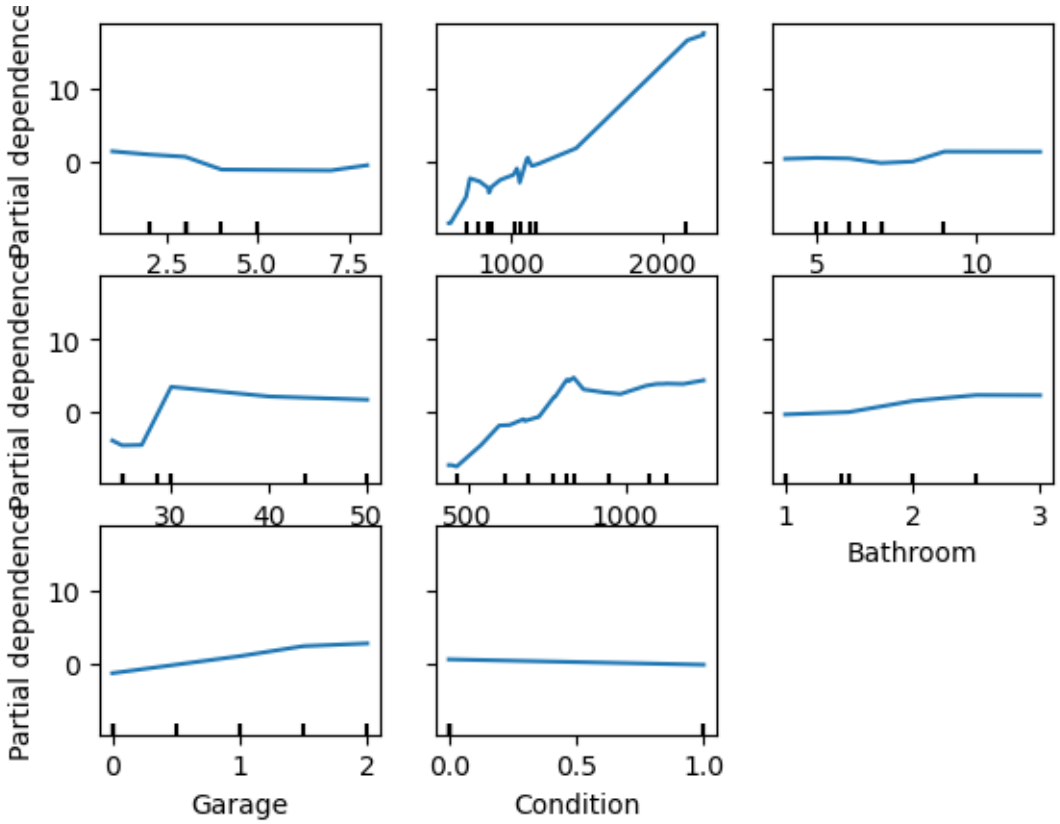
Verimizin lineer olmayan ilişkilere sahip olduğu ortada. Çünkü lineer olmayan modeller çok daha iyi sonuç verdi. Bununla beraber Gradient Boosting en iyi sonucu verdi ve %95.6 R^2 ile çok iyi bir başarı yakaladı. Basit doğrusal regresyon modelini kenarda tutarsak en kötü model ise Lasso oldu. Aslında kısaca lineer olmayan ilişkileri de görebilen ensemble kullanan modeller en iyi sonuçları verdi diyebiliriz.

5.2 En İyi Modeli (Gradient Boosting) Yorumlayalım

[3]:
```
from sklearn.inspection import PartialDependenceDisplay
```

[26]:
```
features = ["Bedroom","Space","Room","Lot","Tax","Bathroom","Garage","Condition"]
```

[28]:
```
PartialDependenceDisplay.from_estimator(gb_best_model,
                                        x_test,
                                        features = features,
                                       kind = "average",
                                       grid_resolution = 50)
mt.show()
```



Bu grafikler ile en iyi tahmin yapan modelimizin (GradientBoosting) nasıl tahminler yaptığını görebiliriz.

- Yatak odası sayısının fiyat üzerinde güçlü bir etkisi bulunmamaktadır. Hatta yatak odası sayısının artışı biraz da olsa fiyatlara negatif etki etmiş.
- Space değişkeni (metrekare) fiyatı en güçlü artıran değişkendir. Metrekare arttıkça fiyatlar çok ciddi bir artış yaşamıştır.
- Oda sayısının ise fiyatlar üstündeki etkisi sınırlıdır.
- Arsa büyüklüğü(Lot), başlangıçta fiyatı artırmakta ancak belirli bir seviyeden sonra etkisi azalmaktadır.
- Vergi değeri (Tax) arttıkça ev fiyatı da artmaktadır. Biraz dalgalı bir şekilde fakat bu değişken fiyat değişkeni ile pozitif ilişkilidir.
- Banyo sayısı arttıkça ev fiyatlarının da belirli düzeyde arttığı görülmüştür.
- Garaj sayısı arttıkça ev fiyatının direkt olarak arttığını görüyoruz.
- Evin genel durumu(Condition) modelde fiyat üzerinde sınırlı etkiye sahiptir.

5.3 Son Çıkarım

En iyi modelimizin sonuçlarına göre, ev fiyatını en çok etkileyen değişkenin metrekare(Space) değişkeni olduğunu gördük. Diğer değişkenlerin etkisi daha sınırlıdır. Özellikle oda sayısı ve evin genel durumu gibi değişkenlerin modele katkısı düşük kalmıştır.